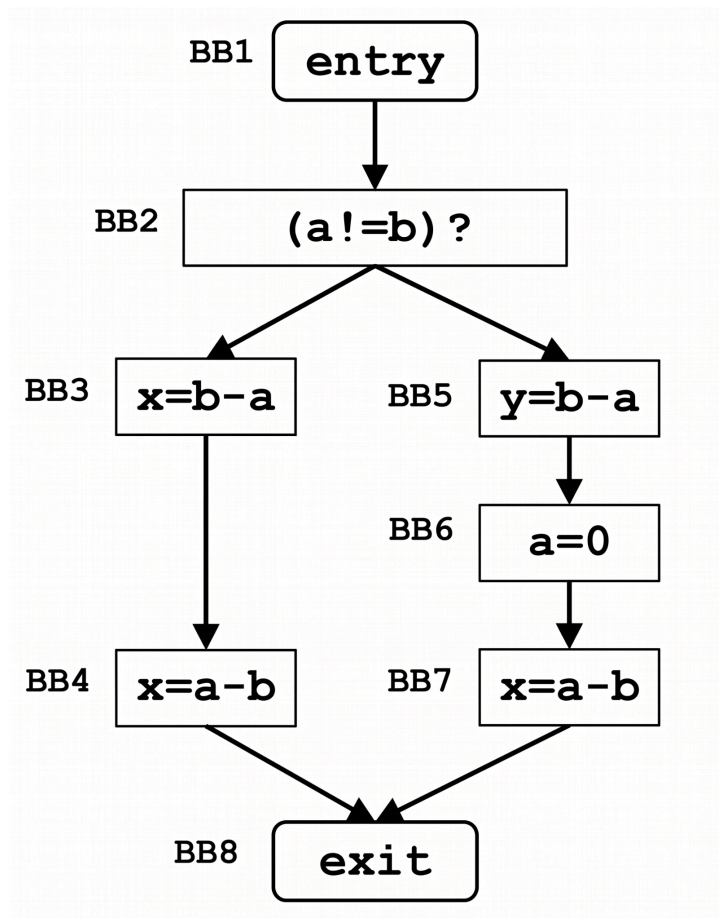


Very Busy Expressions



	Dataflow Problem: Very Busy Expression
Domain	Set di Espressioni
Direction	Backward: $in[b] = fb(out[b])$ $out[b] = \wedge in[succ(b)]$
Transfer function	$in[b] = fb(out[b]) = Gen(b) \cup (out[b] - Kill[b])$
Meet Operator	$\cap$
Boundary Condition	$in[exit] = \emptyset$
Initial Interior Points	$in[b] = u$ (Insieme universo) $u = \{tutte\ le\ espressioni\ del\ CFG\}$

Considerazioni per modellare il problema proposto:

Domain: Set di espressioni

- **Ragionamento:** In questa analisi l'obiettivo non è tracciare nodi o definizioni di variabili, bensì le **espressioni matematiche** (es. $a-b$, $k*2$) valutate nel codice. L'informazione che viene propagata è un insieme di espressioni di cui si vuole verificare la proprietà di essere "Very Busy", ovvero destinate a essere calcolate in **ogni possibile percorso futuro** senza che i loro operandi subiscano alterazioni.

Direction: Backward

- **Ragionamento:** La scelta della direzione è intrinseca alla definizione formale del problema. Per stabilire se un'espressione sarà valutata nel futuro (cioè **da un determinato punto p verso l'uscita EXIT**), è necessario raccogliere le informazioni a partire dal termine del programma e propagarle **a ritroso verso l'inizio (ENTRY)**. Di conseguenza, l'analisi procede **dal basso verso l'alto**, e i dati in ingresso a un blocco ($in[b]$) vengono calcolati a partire dai dati provenienti dai suoi successori ($out[b]$).

Transfer Function: $fb(out[b]) = Gen[b] \cup (out[b] - Kill[b])$

- **Ragionamento:** Lavorando all'indietro, la funzione di trasferimento calcola le espressioni attive all'ingresso del blocco ($in[b]$) partendo da quelle in uscita ($out[b]$).
 - **Kill[b]:** Rappresenta l'**insieme delle espressioni che vengono "uccise" nel blocco corrente**. Se un blocco ridefinisce (sovrascrive) una variabile che funge da operando per un'espressione, quell'espressione perde validità da quel punto risalendo verso l'alto, **poiché il suo valore sarebbe differente**.
 - **Gen[b]:** Rappresenta le **espressioni che vengono fisicamente valutate all'interno del blocco b prima di qualsiasi eventuale ridefinizione dei loro operandi**. L'attraversamento del blocco genera la proprietà "Very Busy" per queste espressioni nei confronti dei blocchi precedenti.
 - **($out[b] - Kill[b]$):** Rappresenta l'insieme delle espressioni che **sopravvivono all'attraversamento del blocco a ritroso**. Indica **le espressioni che risultano Very Busy nei percorsi futuri** ($out[b]$) e di cui il blocco corrente **non ha alterato gli operandi**. Queste espressioni mantengono la loro validità e vengono lasciate passare, continuando a propagarsi verso l'alto.

Meet Operation: Intersezione ($\cap$)

- **Ragionamento:** Dal momento che un'espressione, per essere considerata very busy, **deve essere obbligatoriamente valutata in tutti i percorsi futuri possibili**, l'operatore di meet deve riflettere esattamente questa necessità. Considerando ad esempio una situazione con un costrutto **if**, durante la risalita dal basso del Control Flow Graph **ci si ritrova a dover unire le informazioni provenienti dai due rami inferiori**. In questo punto di snodo è fondamentale conservare esclusivamente le espressioni che sopravvivono **da entrambe le direzioni**. L'operatore matematico che ci permette di fare questa cosa è l'**intersezione**.

Boundary Condition: $in[EXIT] = \emptyset$

- **Ragionamento:** Il nodo EXIT è un blocco che rappresenta la terminazione assoluta del programma. Oltre questo punto non avviene alcuna computazione, pertanto l'insieme delle espressioni matematiche che verranno valutate dopo l'uscita è, per definizione, l'insieme vuoto.

Initial interior points: $in[b]$ = Insieme Universo

- **Ragionamento:** La scelta del valore di inizializzazione per i nodi interni è **dettata dall'operatore di Meet**. Poiché si utilizza l'intersezione, inizializzare i blocchi all'insieme vuoto porterebbe l'algoritmo a una stasi, in quanto **l'intersezione di un qualsiasi insieme con l'insieme vuoto produce sempre l'insieme vuoto** ($\emptyset \cap x = \emptyset$). Inizializzando invece tutti i blocchi interni all'Insieme Universo (**insieme delle espressioni distinte presenti nell'intero CFG**), si consente alle iterazioni di "scremare" progressivamente le espressioni non valide. Questo garantisce all'algoritmo iterativo di convergere in modo sicuro e corretto.

Sezione calcoli:

Dati di partenza noti:

$u = \{a!=b, b-a, a-b\}$

$in[BB1]=in[BB2]=in[BB3]=in[BB4]=in[BB5]=in[BB6]=in[BB7]=in[BB8] = u$

$in[BB8] = \emptyset \quad (in[EXIT] = \emptyset)$

BasicBlock	Gen (b)	Kill (b)
BB1 (Entry)	{ }	{ }
BB2	{ $a!=b$ }	{ }
BB3	{ $b-a$ }	{ }
BB4	{ $a-b$ }	{ }
BB5	{ $b-a$ }	{ }
BB6	{ }	{ $a-b, b-a, a!=b$ }
BB7	{ $a-b$ }	{ }
BB8 (Exit)	{ }	{ }

Formule fondamentali:

- $out[b] = \cap in[succ(b)]$
- $in[b] = Gen(b) \cup (out[b] - Kill(b))$

Iterazione 1:

$in[BB8] = \emptyset$

$out[BB7] = in[BB8] = \emptyset$

$in[BB7] = \{a-b\} \cup (\emptyset - \emptyset) = \{a-b\}$

$out[BB6] = in[BB7] = \{a-b\}$

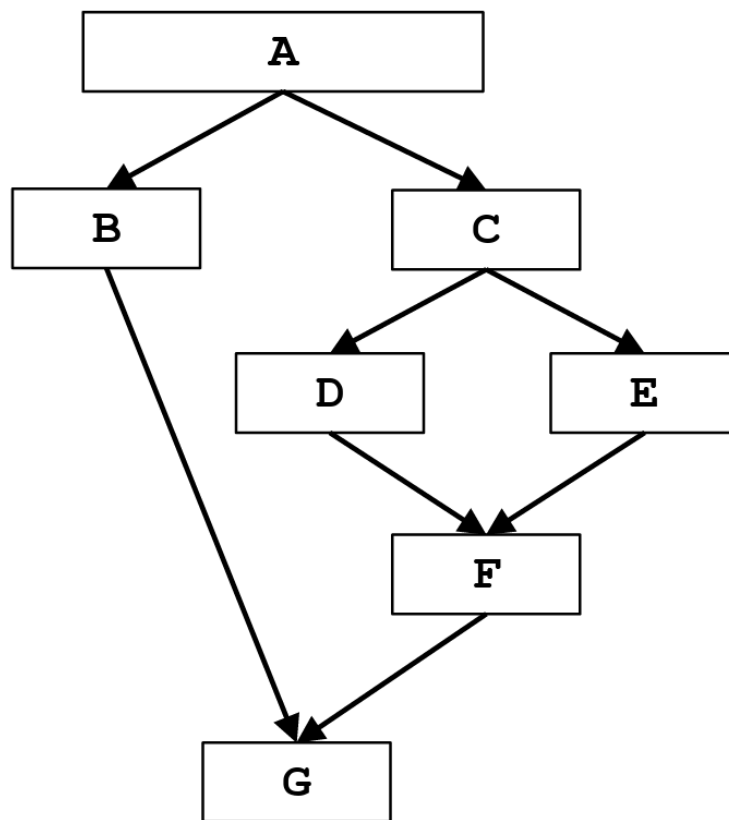
$$\begin{aligned}
in[BB6] &= \emptyset \cup (\{a-b\} - \{a-b, b-a, a!=b\}) = \emptyset \cup \emptyset = \emptyset \\
out[BB5] &= in[BB6] = \emptyset \\
in[BB5] &= \{b-a\} \cup (\emptyset - \emptyset) = \{b-a\} \\
out[BB4] &= in[BB8] = \emptyset \\
in[BB4] &= \{a-b\} \cup (\emptyset - \emptyset) = \{a-b\} \\
out[BB3] &= in[BB4] = \{a-b\} \\
in[BB3] &= \{b-a\} \cup (\{a-b\} - \emptyset) = \{b-a, a-b\} \\
out[BB2] &= in[BB3] \cap in[BB5] = \{b-a, a-b\} \cap \{b-a\} = \{b-a\} \\
in[BB2] &= \{a!=b\} \cup (\{b-a\} - \emptyset) = \{a!=b, b-a\} \\
out[BB1] &= in[BB2] = \{a!=b, b-a\} \\
in[BB1] &= \emptyset \cup (\{a!=b, b-a\} - \emptyset) = \{a!=b, b-a\}
\end{aligned}$$

Considerazioni finali:

Iterazione 1 = Iterazione 2 = Iterazione 3. Dal momento che il CFG è un **DAG (Directed Acyclic Graph) privo di loop**, possiamo quindi assicurare che l'algoritmo **converge in un singolo passaggio**. Data l'**assenza di backedge**, si può garantire infatti che l'informazione propagata bottom-up non subisca ulteriori mutazioni, rendendo le iterazioni successive puramente di controllo.

	Iterazione 1		Iterazione 2		Iterazione 3	
	IN[B]	OUT[B]	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1	{a!=b, b-a}	{a!=b, b-a}	{a!=b, b-a}	{a!=b, b-a}	{a!=b, b-a}	{a!=b, b-a}
BB2	{a!=b, b-a}	{b-a}	{a!=b, b-a}	{b-a}	{a!=b, b-a}	{b-a}
BB3	{b-a, a-b}	{a-b}	{b-a, a-b}	{a-b}	{b-a, a-b}	{a-b}
BB4	{a-b}	{}	{a-b}	{}	{a-b}	{}
BB5	{b-a}	{}	{b-a}	{}	{b-a}	{}
BB6	{}	{a-b}	{}	{a-b}	{}	{a-b}
BB7	{a-b}	{}	{a-b}	{}	{a-b}	{}
BB8	{}	{}	{}	{}	{}	{}

Dominator Analysis



	Dataflow Problem: Dominator Analysis
Domain	Set di Dominatori/Nodi
Direction	Forward: $out[b] = fb(in[b])$ $in[b] = \wedge out[pred(b)]$
Transfer function	$out[b] = fb(in[b]) = Gen(b) \cup (in[b] - Kill[bKill[b]])$ $= \{b\} \cup (in[b] - \emptyset)$ $= \{b\} \cup in[b]$
Meet Operator	$\cap$
Boundary Condition	$out[A] = \{A\}$
Initial Interior Points	$out[b] = u$ (Insieme universo) $u = \{\text{tutti i nodi del CFG}\}$

Considerazioni per modellare il problema proposto:

Domain: Set di dominatori/nodi

- **Ragionamento:** l'informazione propagata è costituita dai blocchi stessi del Control Flow Graph. L'obiettivo è determinare, per ogni blocco, l'insieme dei nodi (i dominatori) che devono essere obbligatoriamente attraversati per raggiungerlo.

Direction: Forward

- **Ragionamento:** La scelta della direzione deriva direttamente dalla definizione formale. Poiché un nodo X domina un nodo Y se X appare in ogni percorso che va **dal blocco ENTRY** al blocco Y, l'analisi deve seguire il naturale flusso di esecuzione del programma, propagando le informazioni top-down, **da ENTRY verso EXIT**.

Transfer Function: $fb(in[b]) = Gen(b) \cup (in[b] - Kill[b])$

- **Ragionamento:** La funzione di trasferimento modella l'effetto del passaggio attraverso un Basic Block 'b' sui dati in ingresso ($in[b]$).
 - **Kill[b] = $\emptyset$:** Il passaggio attraverso un blocco non invalida né cancella lo status di dominatore dei nodi precedenti, quindi possiamo considerarlo sempre uguale a $\emptyset$.
 - **GEN[b] = {b}:** Per definizione, un nodo domina sé stesso. Pertanto, **l'attraversamento del blocco b "genera" la dominanza di b stesso per tutti i nodi successivi**.
 - Unendo i due concetti nella formula standard otteniamo la funzione finale in cui i dominatori in uscita sono l'unione tra il blocco corrente e i dominatori in ingresso:
 - ovvero: $\{b\} \cup in[b]$

Meet Operation: Intersezione ($\cap$)

- **Ragionamento:** La definizione richiede che il nodo dominatore appaia in **OGNI percorso** del grafo che porta al nodo bersaglio. Quando due o più percorsi di controllo confluiscono in un blocco (es. dopo un costrutto if), dobbiamo estrarre solo i dominatori che sono comuni a tutti i percorsi in ingresso. L'operatore matematico che ci permette di fare questa cosa è l'**intersezione**.

Boundary Condition: $out[A] = \{A\}$

- **Ragionamento:** Il blocco **A** è il **blocco ENTRY** ed è il punto di partenza senza predecessori nel CFG. Poiché la regola stabilisce che ogni blocco domina sé stesso, l'unico dominatore valido all'uscita del blocco iniziale è il blocco A (Entry) stesso.

Initial interior points: out[b] = Insieme Universo

- **Ragionamento:** L'inizializzazione dei nodi interni dipende strettamente dall'operatore di Meet. Utilizzando l'intersezione, inizializzare i blocchi all'insieme vuoto porterebbe l'algoritmo a produrre sempre insiemi vuoti ($\emptyset \cap x = \emptyset$), situazione incontrata anche con l'algoritmo di **available expression**. Inizializzando l'out di ogni blocco all'Insieme Universo (**la totalità dei nodi del CFG**), permettiamo alle iterazioni di "scremare" e rimuovere progressivamente i nodi non validi tramite l'intersezione, garantendo la convergenza corretta verso il massimo punto fisso **senza perdita di informazioni dovuta a $\emptyset$** .

Sezione calcoli:

Dati di partenza noti:

- $u = \{A, B, C, D, E, F, G\}$
- $out[B] = out[C] = out[D] = out[E] = out[F] = out[G] = u$
- $out[A] = \{A\}$

BasicBlock	Gen (b)	Kill (b)
A	{A}	{}
B	{B}	{}
C	{C}	{}
D	{D}	{}
E	{E}	{}
F	{F}	{}
G	{G}	{}

Formule fondamentali:

- $in[b] = \cap out[pred(b)]$
- $out[b] = \{b\} \cup in[b]$

Iterazione 1:

```
out[A] = {A}
in[B] = out[A] = {A}
out[B] = {B} ∪ {A} = {A, B}
in[C] = out[A] = {A}
out[C] = {C} ∪ {A} = {A, C}
in[D] = out[C] = {A, C}
out[D] = {D} ∪ {A, C} = {A, C, D}
in[E] = out[C] = {A, C}
out[E] = {E} ∪ {A, C} = {A, C, E}
in[F] = out[D] ∩ out[E] = {A, C, D} ∩ {A, C, E} = {A, C}
out[F] = {F} ∪ {A, C} = {A, C, F}
in[G] = out[B] ∩ out[F] = {A, B} ∩ {A, C, F} = {A}
out[G] = {G} ∪ {A} = {A, G}
```

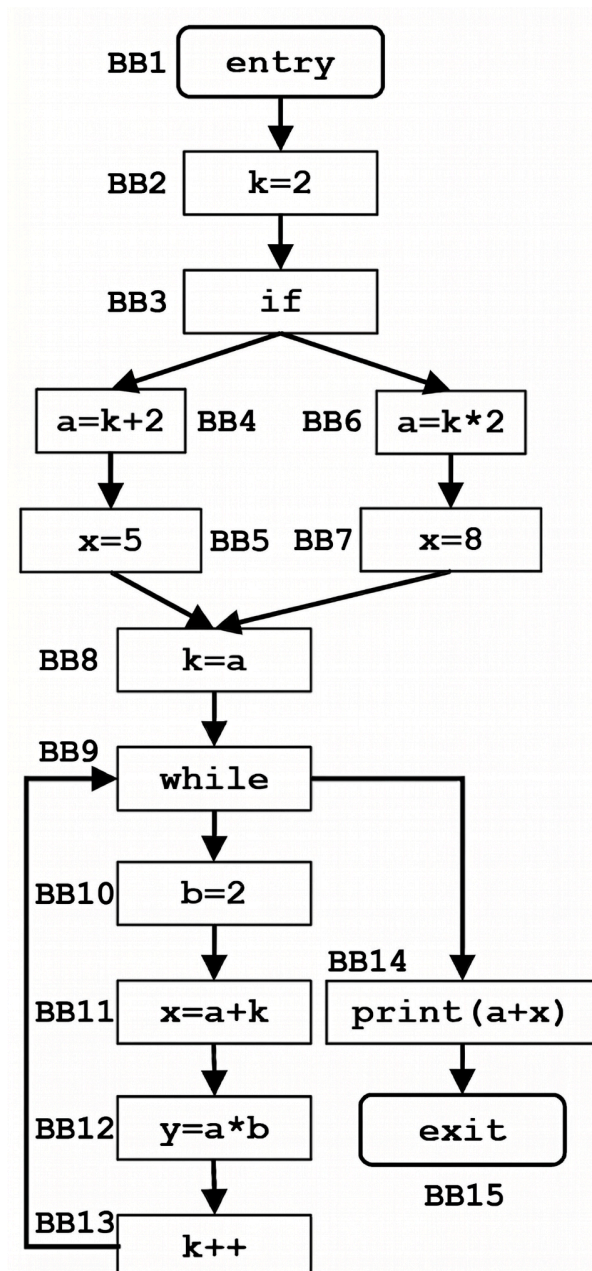
Considerazioni finali:

Come per il problema delle Very Busy Expression, iterazione 1 = iterazione 2 = iterazione 3. Dal momento che il CFG è un DAG (**Directed Acyclic Graph**) **privo di loop**, possiamo quindi assicurare che **l'algoritmo converge in un singolo passaggio**.

Data l'**assenza di backedge**, si può garantire infatti che l'informazione propagata top-down non subisca ulteriori mutazioni, rendendo le iterazioni successive puramente di controllo.

	Iterazione 1		Iterazione 2		Iterazione 3	
	IN[B]	OUT[B]	IN[B]	OUT[B]	IN[B]	OUT[B]
A		{A}		{A}		{A}
B	{A}	{A, B}	{A}	{A, B}	{A}	{A, B}
C	{A}	{A, C}	{A}	{A, C}	{A}	{A, C}
D	{A, C}	{A, C, D}	{A, C}	{A, C, D}	{A, C}	{A, C, D}
E	{A, C}	{A, C, E}	{A, C}	{A, C, E}	{A, C}	{A, C, E}
F	{A, C}	{A, C, F}	{A, C}	{A, C, F}	{A, C}	{A, C, F}
G	{A}	{A, G}	{A}	{A, G}	{A}	{A, G}

Constant Propagation



	Dataflow Problem: Constant Propagation
Domain	Set di Coppie <Variabile, Valore Costante>
Direction	Forward: $out[b] = fb(in[b])$ $in[b] = \wedge out[pred(b)]$
Transfer function	$out[b] = fb(in[b]) = Gen(b) \cup (in[b] - Kill[b])$

Meet Operator	$\cap$
Boundary Condition	$\text{out}[\text{entry}] = \emptyset$
Initial Interior Points	$\text{out}[b] = u$ (Insieme universo) $u = \{\text{tutte le coppie } \langle \text{var}, \text{val} \rangle \text{ del CFG}\}$

Considerazioni per modellare il problema proposto:

Domain: Set di coppie <variabile, valore costante>

- **Ragionamento:** In questa analisi l'obiettivo non è tracciare i percorsi o le espressioni, bensì determinare in quali punti del programma le variabili assumono un **valore numerico fisso e prevedibile** a tempo di compilazione. L'informazione che viene propagata è quindi l'associazione diretta tra il nome della variabile e il suo valore esatto.

Direction: Forward

- **Ragionamento:** La scelta della direzione deriva dalla natura delle variabili. Il valore costante di una variabile viene stabilito nel momento in cui avviene un'assegnazione e si propaga in avanti seguendo il naturale flusso di esecuzione del programma, fino a quando la variabile non viene riassegnata. L'analisi procede quindi **dall'alto verso il basso** (top-down), calcolando i dati in uscita ($\text{out}[b]$) a partire da quelli in ingresso ($\text{in}[b]$).

Transfer Function: $\text{out}[b] = \text{Gen}[b] \cup (\text{in}[b] - \text{Kill}[b])$

- **Ragionamento:** Lavorando in avanti, la funzione di trasferimento calcola le costanti in uscita dal blocco partendo da quelle in ingresso.
 - **Kill[b]:** Rappresenta l'insieme delle associazioni precedenti che vengono invalidate ("uccise") dalle istruzioni del blocco corrente. **Quando un blocco sovrascrive una variabile, è necessario rimuovere qualsiasi vecchio valore a essa associato** prima di poterne generare uno nuovo. Per questo motivo, l'operazione di Kill ha come bersaglio direttamente il nome della variabile (es. $\{k\}$). Ci permette di evitare situazioni in cui, al termine del blocco, con la stessa variabile ci siano assegnati contemporaneamente due valori costanti differenti.
 - **Gen[b]:** Rappresenta le **nuove assegnazioni a valori costanti** effettuate all'interno del blocco. Questo include anche i risultati di espressioni matematiche in cui tutti gli operandi sono a loro volta costanti note in quel punto.
 - **(in[b] - Kill[b]):** Rappresenta l'insieme delle costanti che **sopravvivono** all'attraversamento del blocco. Queste costanti non sono state alterate dalle istruzioni del blocco corrente e possono continuare a propagarsi verso il basso, unendosi a quelle appena generate.

Meet Operation: Intersezione($\cap$)

- **Ragionamento:** Affinché una variabile possa essere considerata una costante garantita in un punto di convergenza del Control Flow Graph (ad esempio dopo un costrutto if), essa deve obbligatoriamente possedere lo **stesso identico valore in tutti i percorsi in ingresso**. Se i valori differiscono tra un ramo e l'altro, o se la variabile è costante solo in un ramo, la proprietà di costanza si perde. L'operatore di intersezione applicato alle coppie modella esattamente questo vincolo.

Boundary Condition: out[ENTRY] = $\emptyset$

- **Ragionamento:** Il blocco ENTRY rappresenta il punto di inizio assoluto del programma. In questa fase iniziale nessuna variabile è ancora stata dichiarata o ha ricevuto un'assegnazione. Di conseguenza, l'insieme delle costanti disponibili in uscita dal blocco iniziale è, per definizione, **l'insieme vuoto**.

Initial interior points: Insieme Universo

- **Ragionamento:** La scelta di questo valore di inizializzazione per i nodi interni nasce da una doppia necessità logica e pratica. Innanzitutto, è un obbligo dettato dall'operatore di Meet.
Dato che utilizziamo l'intersezione, inizializzare i blocchi all'insieme vuoto porterebbe l'algoritmo a uno **stallo immediato**, in quanto l'intersezione di un qualsiasi insieme con il vuoto produce sempre il vuoto ($\emptyset \cap x = \emptyset$).
Per questo assegniamo l'Insieme Universo (U). A livello pratico, U rappresenta l'inventario completo di **tutte le possibili associazioni <variabile, valore> generabili nel programma**. Affinché questo inventario rispetti rigidamente il nostro Dominio (che ammette **solo valori numerici e non espressioni**), **le istruzioni matematiche sono state valutate a priori basandosi sui valori "sicuri" noti alla prima iterazione** (ad esempio, inserendo direttamente la coppia <a, 4> al posto dell'espressione $a=k+2$). Questa assunzione iniziale, totalmente ottimistica, funge da "paracadute" per la prima iterazione evitando così che le costanti vengano distrutte al primo incrocio di percorsi (come nell'header del while) e permette alle iterazioni successive di far collidere i dati reali, "scremando" progressivamente le costanti discordanti per convergere in modo sicuro verso la soluzione corretta.

Sezione calcoli:

Dati di partenza noti:

- $u = \{ \langle k, 2 \rangle, \langle a, 4 \rangle, \langle x, 5 \rangle, \langle x, 8 \rangle, \langle k, 4 \rangle, \langle b, 2 \rangle, \langle y, 8 \rangle, \langle k, 5 \rangle \}$
- $out[BB2] = out[BB3] = out[BB4] = out[BB5] = out[BB6] = out[BB7] =$
 $out[BB8] = out[BB9] = out[BB10] = out[BB11] = out[BB12] = out[BB13] = out[BB14] =$
 $out[BB15] = u$
- $out[BB1] = \emptyset$ ($out[entry] = \emptyset$)

BasicBlock	Gen(b) per i-esima iter.	Kill(b) per i-esima iter.
BB1	{ }	{ }
BB2	{<k,2>}	{ }
BB3	{ }	{ }
BB4	{<a, val (k) +2>}	{a}
BB5	{<x,5>}	{x}
BB6	{<a, val (k) *2>}	{a}
BB7	{<x,8>}	{x}
BB8	{<k, val (a) >}	{k}
BB9	{ }	{ }
BB10	{<b,2>}	{b}
BB11	{<x, val (a) +val (k) >}	{x}
BB12	{<y, val (a) *val (b) >}	{y}
BB13	{k, val (k) +1}	{k}
BB14	{ }	{ }
BB15	{ }	{ }

Formule fondamentali:

- $in[b] = \cap out(prec(b))$
- $out[b] = Gen(b) \cup (in[b] - Kill(b))$

Appunto per calcoli di ogni iterazione:

- Ad ogni i-esima iterazione sostituisco la generica espressione che rappresenta il valore nella coppia <variabile, valore> (es. <y, val (a) *val (b) >) con i valori corrispondenti alle variabili che assumono nell' iterazione corrente.

Iterazione 1:

$out[BB1] = \emptyset$

$in[BB2] = out[BB1] = \emptyset$

$out[BB2] = \{<k,2>\} \cup (\emptyset - \{k\}) = \{<k,2>\}$

$in[BB3] = out[BB2] = \{<k,2>\}$

$out[BB3] = \emptyset \cup (\{<k,2>\} - \emptyset) = \{<k,2>\}$

$in[BB4] = out[BB3] = \{<k,2>\}$

$out[BB4] = \{<a,4>\} \cup (\{<k,2>\} - \{a\}) = \{<a,4>, <k,2>\}$

$in[BB5] = out[BB4] = \{<a,4>, <k,2>\}$

$out[BB5] = \{<x,5>\} \cup (\{<a,4>, <k,2>\} - \{x\}) = \{<x,5>, <a,4>, <k,2>\}$

$\text{in}[\text{BB6}] = \text{out}[\text{BB3}] = \{\langle k, 2 \rangle\}$
 $\text{out}[\text{BB6}] = \{\langle a, 4 \rangle\} \cup (\{\langle k, 2 \rangle\} - \{a\}) = \{\langle a, 4 \rangle, \langle k, 2 \rangle\}$

$\text{in}[\text{BB7}] = \text{out}[\text{BB6}] = \{\langle a, 4 \rangle, \langle k, 2 \rangle\}$
 $\text{out}[\text{BB7}] = \{\langle x, 8 \rangle\} \cup (\{\langle a, 4 \rangle, \langle k, 2 \rangle\} - \{x\}) = \{\langle x, 8 \rangle, \langle a, 4 \rangle, \langle k, 2 \rangle\}$

$\text{in}[\text{BB8}] = \text{out}[\text{BB5}] \cap \text{out}[\text{BB7}] = \{\langle x, 5 \rangle, \langle a, 4 \rangle, \langle k, 2 \rangle\} \cap \{\langle x, 8 \rangle, \langle a, 4 \rangle, \langle k, 2 \rangle\} =$
 $= \{\langle a, 4 \rangle, \langle k, 2 \rangle\}$
 $\text{out}[\text{BB8}] = \{\langle k, 4 \rangle\} \cup (\{\langle a, 4 \rangle, \langle k, 2 \rangle\} - \{k\}) = (\{k\} \text{ killa la coppia precedente } \langle k, 2 \rangle)$
 $= \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB9}] = \text{out}[\text{BB8}] \cap \text{out}[\text{BB13}] = \{\langle k, 4 \rangle, \langle a, 4 \rangle\} \cap u =$
 $= \{\langle k, 4 \rangle, \langle a, 4 \rangle\} \cap \{\langle k, 2 \rangle, \langle a, 4 \rangle, \langle x, 5 \rangle, \langle x, 8 \rangle, \langle k, 4 \rangle, \langle b, 2 \rangle, \langle y, 8 \rangle, \langle k, 5 \rangle\} =$
 $= \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB9}] = \emptyset \cup (\{\langle k, 4 \rangle, \langle a, 4 \rangle\} - \emptyset) = \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB10}] = \text{out}[\text{BB9}] = \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB10}] = \{\langle b, 2 \rangle\} \cup (\{\langle k, 4 \rangle, \langle a, 4 \rangle\} - \{b\}) = \{\langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB11}] = \text{out}[\text{BB10}] = \{\langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB11}] = \{\langle x, 8 \rangle\} \cup (\{\langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\} - \{x\}) =$
 $= \{\langle x, 8 \rangle, \langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB12}] = \text{out}[\text{BB11}] = \{\langle x, 8 \rangle, \langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB12}] = \{\langle y, 8 \rangle\} \cup (\{\langle x, 8 \rangle, \langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\} - \{y\}) =$
 $= \{\langle y, 8 \rangle, \langle x, 8 \rangle, \langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB13}] = \text{out}[\text{BB12}] = \{\langle y, 8 \rangle, \langle x, 8 \rangle, \langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB13}] = \{k, 5\} \cup (\{\langle y, 8 \rangle, \langle x, 8 \rangle, \langle b, 2 \rangle, \langle k, 4 \rangle, \langle a, 4 \rangle\} - \{k\}) =$
 $= \{\langle k, 5 \rangle, \langle y, 8 \rangle, \langle x, 8 \rangle, \langle b, 2 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB14}] \text{ (condizione falsa del while)} = \text{out}[\text{BB9}] = \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB14}] = \emptyset \cup (\{\langle k, 4 \rangle, \langle a, 4 \rangle\} - \emptyset) = \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$

$\text{in}[\text{BB15}] = \text{out}[\text{BB14}] = \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$
 $\text{out}[\text{BB15}] = \emptyset \cup (\{\langle k, 4 \rangle, \langle a, 4 \rangle\} - \emptyset) = \{\langle k, 4 \rangle, \langle a, 4 \rangle\}$

Iterazione 2:

$\text{in}[\text{BB1}] = \emptyset$
 $\text{out}[\text{BB1}] = \emptyset$

$\text{in}[\text{BB2}] = \emptyset$
 $\text{out}[\text{BB2}] = \{\langle k, 2 \rangle\}$

$\text{in}[\text{BB3}] = \{\langle k, 2 \rangle\}$
 $\text{out}[\text{BB3}] = \{\langle k, 2 \rangle\}$

$\text{in}[\text{BB4}] = \{\langle k, 2 \rangle\}$
 $\text{out}[\text{BB4}] = \{\langle a, 4 \rangle, \langle k, 2 \rangle\}$

```
in[BB5] = {<a,4>,<k,2>}
out[BB5] = {<x,5>,<a,4>,<k,2>}
```

```
in[BB6] = {<k,2>}
out[BB6] = {<a,4>,<k,2>}
```

```
in[BB7] = {<a,4>,<k,2>}
out[BB7] = {<x,8>,<a,4>,<k,2>}
```

```
in[BB8] = {<a,4>,<k,2>}
out[BB8] = {<k,4>,<a,4>}
```

(Nota: Comunico al compilatore che **k ha perso la garanzia di possedere un valore costante predicibile**, il compilatore "rinuncia" a ottimizzare e pre-calcolare il risultato. Di conseguenza, per tutte le istruzioni successive che dipendono da essa (come $x = a + k$), l'insieme Gen risulterà $\emptyset$)

```
in[BB9] = out[BB8] ∩ out[BB13] =
= {<k,4>,<a,4>} ∩ {<k,5>,<y,8>,<x,8>,<b,2>,<a,4>} = {<a,4>}
out[BB9] = ∅ ∪ ({<a,4>} - ∅) = {<a,4>}
```

```
in[BB10] = out[BB9] = {<a,4>}
out[BB10] = {<b,2>} ∪ ({<a,4>} - {b}) = {<b,2>,<a,4>}
```

```
in[BB11] = out[BB10] = {<b,2>,<a,4>}
out[BB11] = ∅ ∪ ({<b,2>,<a,4>} - ∅) = {<b,2>,<a,4>} (Nota: il Gen è vuoto perché per calcolare x serve k, che in questo punto non è più nota)
```

```
in[BB12] = out[BB11] = {<b,2>,<a,4>}
out[BB12] = {<y,8>} ∪ ({<b,2>,<a,4>} - {y}) = {<y,8>,<b,2>,<a,4>}
```

```
in[BB13] = out[BB12] = {<y,8>,<b,2>,<a,4>}
out[BB13] = ∅ ∪ ({<y,8>,<b,2>,<a,4>} - {k}) = {<y,8>,<b,2>,<a,4>} (Nota: il Gen è vuoto perché l'incremento si basa su k, non più nota)
```

```
in[BB14] (condizione falsa del while) = out[BB9] = {<a,4>}
out[BB14] = ∅ ∪ ({<a,4>} - ∅) = {<a,4>}
```

```
in[BB15] = out[BB14] = {<a,4>}
out[BB15] = ∅ ∪ ({<a,4>} - ∅) = {<a,4>}
```

Iterazione 3:

in[BB1 ... BB8] e out[BB1 ... BB8] rimangono invariati

```
in[BB9] = out[BB8] ∩ out[BB13] =
{<k,4>,<a,4>} ∩ {<k,5>,<y,8>,<x,8>,<b,2>,<a,4>} = {<a,4>}
out[BB9] = ∅ ∪ ({<a,4>} - ∅) = {<a,4>}
```

anche in[BB10 ... BB15] e out[BB10 ... BB15] rimangono invariati

Considerazioni finali:

L'algoritmo iterativo ha raggiunto il suo **Punto Fisso** alla terza iterazione, dimostrando la corretta convergenza del Dataflow Framework.

Come evidenziato dai calcoli, la **Constant Propagation si conferma un'analisi strettamente conservativa**, infatti nell'header del ciclo (BB9), la discordanza dei valori di k (4 proveniente dal ramo sequenziale, 5 dal backedge) ha portato alla **rimozione della variabile dall'insieme delle costanti**.

Questo comportamento riflette la natura statica dell'analisi a **tempo di compilazione**, infatti non potendo determinare a priori il flusso di esecuzione a runtime, **il compilatore declassa i valori variabili a non predicibili**, garantendo così l'assoluta sicurezza e correttezza del codice generato.

Di conseguenza, il propagarsi di questa incertezza ha svuotato progressivamente gli insiemi di generazione (Gen(b)) all'interno del loop. L'unica costante che è sopravvissuta a tutti i cicli del Control Flow Graph, e che il compilatore potrà tranquillamente ottimizzare alla fine del programma (in[BB15]), risulta essere **<a, 4>**.

	Iterazione 1		Iterazione 2		Iterazione 3	
	IN[B]	OUT[B]	IN[B]	OUT[B]	IN[B]	OUT[B]
BB1	{}	{}	{}	{}	{}	{}
BB2	{}	{<k, 2>}	{}	{<k, 2>}	{}	{<k, 2>}
BB3	{<k, 2>}	{<k, 2>}	{<k, 2>}	{<k, 2>}	{<k, 2>}	{<k, 2>}
BB4	{<k, 2>}	{<a, 4>, <k, 2>}	{<k, 2>}	{<a, 4>, <k, 2>}	{<k, 2>}	{<a, 4>, <k, 2>}
BB5	{<a, 4>, <k, 2>}	{<x, 5>, <a, 4>, <k, 2>}	{<a, 4>, <k, 2>}	{<x, 5>, <a, 4>, <k, 2>}	{<a, 4>, <k, 2>}	{<x, 5>, <a, 4>, <k, 2>}
BB6	{<k, 2>}	{<a, 4>, <k, 2>}	{<k, 2>}	{<a, 4>, <k, 2>}	{<k, 2>}	{<a, 4>, <k, 2>}
BB7	{<a, 4>, <k, 2>}	{<x, 8>, <a, 4>, <k, 2>}	{<a, 4>, <k, 2>}	{<x, 8>, <a, 4>, <k, 2>}	{<a, 4>, <k, 2>}	{<x, 8>, <a, 4>, <k, 2>}
BB8	{<a, 4>, <k, 2>}	{<k, 4>, <a, 4>}	{<a, 4>, <k, 2>}	{<k, 4>, <a, 4>}	{<a, 4>, <k, 2>}	{<k, 4>, <a, 4>}
BB9	{<k, 4>, <a, 4>}	{<k, 4>, <a, 4>}	{<a, 4>}	{<a, 4>}	{<a, 4>}	{<a, 4>}
BB10	{<k, 4>, <a, 4>}	{<b, 2>, <k, 4>, <a, 4>}	{<a, 4>}	{<b, 2>, <a, 4>}	{<a, 4>}	{<b, 2>, <a, 4>}
BB11	{<b, 2>, <k, 4>, <a, 4>}	{<x, 8>, <b, 2>, <k, 4>, <a, 4>}	{<b, 2>, <a, 4>}	{<b, 2>, <a, 4>}	{<b, 2>, <a, 4>}	{<b, 2>, <a, 4>}
BB12	{<x, 8>, <a, 4>}	{<y, 8>, <a, 4>}	{<b, 2>, <a, 4>}	{<y, 8>, <a, 4>}	{<b, 2>, <a, 4>}	{<y, 8>, <a, 4>}

	$\langle b, 2 \rangle,$ $\langle k, 4 \rangle,$ $\langle a, 4 \rangle$	$\langle x, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle k, 4 \rangle,$ $\langle a, 4 \rangle$	$\langle a, 4 \rangle$	$\langle b, 2 \rangle,$ $\langle a, 4 \rangle$	$\langle a, 4 \rangle$	$\langle b, 2 \rangle,$ $\langle a, 4 \rangle$
BB13	$\{\langle y, 8 \rangle,$ $\langle x, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle k, 4 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle k, 5 \rangle,$ $\langle y, 8 \rangle,$ $\langle x, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle y, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle y, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle y, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle y, 8 \rangle,$ $\langle b, 2 \rangle,$ $\langle a, 4 \rangle\}$
BB14	$\{\langle k, 4 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$
BB15	$\{\langle k, 4 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle k, 4 \rangle,$ $\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$	$\{\langle a, 4 \rangle\}$